

REACT JS Assignment

Handling Events in React

Question 1: How are events handled in React compared to vanilla JavaScript? Explain the concept of synthetic events.

Answer: In vanilla JavaScript, events are handled by directly attaching event listeners to DOM elements using methods like `addEventListener()`. In React, events are handled using JSX event attributes such as `onClick`, `onChange`, and `onSubmit`.

React uses **Synthetic Events**, which are wrapper objects around the browser's native events. Synthetic events provide:

- **Cross-browser compatibility:** They wrap the differences across browsers so your code works identically everywhere.
- **Consistent behavior:** Properties work the same way in all environments.
- **Improved performance:** React uses a single global event handler for efficiency via event delegation.

Example:

```
function App() {  
  const handleClick = () => {  
    alert("Button Clicked");  
  };  
  return <button onClick={handleClick}>Click Me</button>;  
}
```

Question 2: What are some common event handlers in React.js? Provide examples of `onClick`, `onChange`, and `onSubmit`.

Answer: Common event handlers in React include:

- **onClick** – triggered when an element is clicked.
- **onChange** – triggered when the value of an input field changes.
- **onSubmit** – triggered when a form is submitted.

Examples:

```
// onClick Example  
<button onClick={() => alert("Clicked!")}>Click</button>
```

```
// onChange Example  
<input type="text" onChange={(e) => console.log(e.target.value)} />
```

```
// onSubmit Example  
<form onSubmit={(e) => {  
  e.preventDefault(); // Prevents the page from refreshing
```

```
    alert("Form Submitted");
  }}>
  <button type="submit">Submit</button>
</form>
```

Question 3: Why do you need to bind event handlers in class components?

Answer: In React class components, JavaScript methods are not bound automatically to the class instance. If you pass a method as an event handler callback without binding it, the value of this becomes undefined when the function is eventually called. Binding ensures that this properly refers to the current component instance so you can access this.state or this.props.

Example:

```
import React from 'react';

class App extends React.Component {
  constructor(props) {
    super(props);
    // Explicitly binding the method to the class instance
    this.handleClick = this.handleClick.bind(this);
  }

  handleClick() {
    console.log(this); // Refers to the App component instance
  }

  render() {
    return <button onClick={this.handleClick}>Click</button>;
  }
}
```

LAB EXERCISE

Task 1: Create a button in a React component that changes the text from "Not Clicked" to "Clicked!" when clicked.

```
import React, { useState } from 'react';

function ClickButton() {
  // Initialize state. Default is false (not clicked).
  const [isClicked, setIsClicked] = useState(false);

  return (
    // Update state to true when the button is clicked.
    // Use a ternary operator to conditionally display the text.
    <button onClick={() => setIsClicked(true)}>
      {isClicked ? 'Clicked!' : 'Not Clicked'}
    </button>
  );
}
```

```
);
}
```

```
export default ClickButton;
```

Task 2: Create a form with an input field in React and display the value dynamically as the user types.

```
import React, { useState } from 'react';
```

```
function DynamicForm() {
  // Create a state variable to hold the input text
  const [text, setText] = useState("");

  // Update the state whenever the input changes
  const handleChange = (event) => {
    setText(event.target.value);
  };

  return (
    <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
      <form onSubmit={(e) => e.preventDefault()}>
        <label htmlFor="dynamic-input" style={{ display: 'block', marginBottom: '8px' }}>
          Type something:
        </label>
        <input
          id="dynamic-input"
          type="text"
          value={text} // Ties the input visual value to our state
          onChange={handleChange} // Fires on every single keystroke
          placeholder="Start typing..."
          style={{ padding: '8px', fontSize: '16px', width: '250px' }}
        />
      </form>

      { /* Display the value dynamically right below */ }
      <div style={{ marginTop: '20px' }}>
        <strong>Live Output:</strong>
        <p style={{ fontStyle: text ? 'normal' : 'italic', color: text ? '#000' : '#888' }}>
          {text || 'The output will appear here...'}
        </p>
      </div>
    </div>
  );
}
```

```
export default DynamicForm;
```

2. Conditional Rendering

Question 1: What is conditional rendering in React? How can you conditionally render elements in a React component?

Answer: Conditional rendering in React means displaying different UI elements or components depending on certain conditions (like whether a user is logged in, if data is loading, or if an error occurred). React reads your conditions using standard JavaScript logical structures and outputs the matching JSX.

You can conditionally render elements using:

- if-else statements (outside of JSX)
- Ternary operators (condition ? true : false)
- Logical AND operators (condition && elements)

Question 2: Explain how if-else, ternary operators, and && (logical AND) are used in JSX for conditional rendering.

Answer:

- **if-else statements:** You cannot place an if-else statement directly inside your return block/JSX. Instead, you use them *before* the return block to choose what code executes or which component returns.

// Example:

```
if (isLoggedIn) {  
  return <h1>Welcome back!</h1>;  
} else {  
  return <h1>Please log in.</h1>;  
}
```

- **Ternary Operator (?:):** This is perfect for inline if-else branching directly inside your JSX blocks. It checks the condition: if true, it renders the item after the ?; if false, it renders the item after the :.

// Example:

```
<div>  
  {isLoggedIn ? <button>Logout</button> : <button>Login</button>}  
</div>
```

- **Logical AND (&&):** Use this shortcut when you want an element to render **only if a condition is true**, and render absolutely nothing if it is false.

// Example:

```
<div>  
  {hasNotifications && <p>You have unread messages!</p>}  
</div>
```

LAB EXERCISE

Task 1: Create a component that conditionally displays a login or logout button based on the user's login status.

```
import React, { useState } from 'react';

function AuthButton() {
  // Create a state variable to track if user is logged in (default is false)
  const [isLoggedIn, setIsLoggedIn] = useState(false);

  // Toggles the login state between true and false
  const toggleAuth = () => {
    setIsLoggedIn(!isLoggedIn);
  };

  return (
    <div style={{ padding: '20px', fontFamily: 'sans-serif', textAlign: 'center' }}>

      <h2>{isLoggedIn ? 'Welcome User!' : 'Please log in to continue.'}</h2>

      <button
        onClick={toggleAuth}
        style={{
          padding: '10px 20px',
          fontSize: '16px',
          backgroundColor: isLoggedIn ? '#f44336' : '#4CAF50',
          color: 'white',
          border: 'none',
          borderRadius: '4px',
          cursor: 'pointer'
        }}
      >
        {isLoggedIn ? 'Log Out' : 'Log In'}
      </button>
    </div>
  );
}

export default AuthButton;
```